

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



Báo Cáo Bài Tập Lớn

Nhập môn trí tuệ nhân tạo (CO3061)

Homework 3: GENETIC ALGORITHM
XẾP LỊCH LỚP HỌC SỬ DỤNG THUẬT TOÁN DI
TRUYỀN

Nhóm: L01 - Group 3

GVHD: T.S. Nguyễn Quốc Minh

SVTH: Nguyễn Đình Bằng - 2210298
Trần Minh Quân - 2212822
Phạm Lưu Bang - 2410199
Lê Quang Trường - 2213730
Trần Hữu Luân - 2211954



TP. Hồ Chí Minh, Tháng 5 năm 2026



Phân chia công việc

Thành viên	Công việc	Tỉ lệ đóng góp
Nguyễn Đình Bằng	Thiết kế kiến trúc GA, mã hóa Chromosome và xây dựng hệ thống ràng buộc. (Core GA)	Đóng góp: 100%
Phạm Lưu Bang	Cài đặt các toán tử di truyền (Selection, Crossover, Mutation) và hàm Repair. (GA Operators)	Đóng góp: 100%
Trần Minh Quân	Xây dựng module sinh dữ liệu ngẫu nhiên và quản lý cấu hình hệ thống. (Data & Config)	Đóng góp: 100%
Lê Quang Trưng	Phát triển giao diện Streamlit, trực quan hóa biểu đồ và xuất báo cáo. (UI & Visualization)	Đóng góp: 100%
Trần Hữu Luân	Thực nghiệm trên nhiều seed, đánh giá hiệu năng và hoàn thiện báo cáo. (Evaluation & Docs)	Đóng góp: 100%

Tóm tắt nội dung

Bài toán xếp lịch lớp học (University Course Timetabling) là một bài toán tối ưu tổ hợp thuộc lớp NP-hard với không gian tìm kiếm cực kỳ lớn và nhiều ràng buộc phức tạp. Báo cáo này trình bày việc thiết kế và cài đặt hệ thống xếp lịch tự động sử dụng Thuật toán Di truyền (Genetic Algorithm - GA) để tối ưu hóa việc phân bổ tài nguyên giảng đường và thời gian biểu.

Hệ thống được xây dựng với cấu hình thực tế bao gồm 15 môn học, 10 giảng viên, 30 phòng học (với các loại kích thước khác nhau) và các ràng buộc về thời gian học không liên kề. Chúng tôi đề xuất sử dụng chiến lược lai ghép đồng nhất (Uniform Crossover) kết hợp với kỹ thuật khởi tạo dựa trên heuristic (Heuristic Seeding) để tăng tốc độ hội tụ. Kết quả thực nghiệm trên nhiều bộ tham số ngẫu nhiên (seeds) cho thấy thuật toán luôn tìm được lịch xếp hợp lệ (0 vi phạm ràng buộc cứng) với độ thích nghi cao và phân bổ tải trọng phòng học đồng đều. Hệ thống cũng tích hợp giao diện Streamlit giúp người dùng dễ dàng theo dõi quá trình tiến hóa và xuất kết quả lịch học chi tiết.

Từ khóa: Thuật toán Di truyền, Xếp lịch lớp học, Thỏa mãn ràng buộc, Tối ưu hóa, Trí tuệ Nhân tạo.

Mục lục

1	Giới thiệu	1
1.1	Mục tiêu & Yêu cầu	1
1.2	Động lực thực hiện	1
1.3	Phạm vi của dự án	2
1.4	Cấu trúc bài báo	2
2	Các nghiên cứu liên quan	2
2.1	Bài toán Timetabling	2
2.2	Genetic Algorithm cho Timetabling	3
2.3	Mã hóa Chromosome	3
2.4	Xử lý Ràng buộc trong GA	4
3	Cơ sở lý thuyết	4
3.1	Genetic Algorithm (GA)	4
3.2	Biểu diễn Chromosome cho Timetabling	5
3.3	Hàm Fitness	5
3.4	Tournament Selection	6
3.5	Uniform Crossover	6
3.6	Elitism	6
4	Phương pháp đề xuất	6
4.1	Tổng quan kiến trúc hệ thống	6
4.2	Sinh dữ liệu đầu vào	7
4.2.1	Phòng học	7
4.2.2	Giảng viên và môn học	7
4.2.3	Offerings	7
4.3	Mã hóa Chromosome	8
4.3.1	Cấu trúc Gene	8
4.3.2	Ngày hợp lệ	8
4.3.3	Repair Function	9

4.4	Các toán tử GA	9
4.4.1	Tournament Selection	9
4.4.2	Uniform Crossover	9
4.4.3	Mutation	10
4.5	GA Engine	10
4.5.1	Khởi tạo thông minh (Heuristic Seeding)	10
4.5.2	Cơ chế sửa lỗi (Repair Mechanism)	10
4.5.3	Điều kiện dừng sớm (Early Stopping)	11
4.6	Tham số GA	11
5	Kết quả và Phân tích	11
5.1	Thiết lập Thực nghiệm	11
5.2	Phân tích Hội tụ	12
5.3	Kết quả Multi-seed	12
5.4	Phân tích Tổng hợp	13
5.5	Biểu đồ Fitness	13
5.6	Phân tích Ràng buộc	14
5.7	Kết luận từ Thực nghiệm	15
6	Hạn chế và Thách thức	15
6.1	Hạn chế hiện tại	15
6.2	Thách thức kỹ thuật	16
6.3	Cơ hội cải tiến trong tương lai	16
7	Kết luận và Hướng phát triển	16
7.1	Kết luận	16
7.2	Đóng góp Chính	17
7.3	Hướng phát triển tương lai	17
8	Tài liệu tham khảo	18



Danh sách hình vẽ

1	Biểu đồ quá trình tiến hóa của Fitness và số lượng vi phạm ràng buộc cứng qua các thế hệ.	14
2	So sánh Soft Penalty giữa các seed.	14
3	Số thế hệ cần thiết để hội tụ.	14
4	Phân tích hiệu suất thuật toán trên nhiều seed khác nhau.	14



Danh sách bảng

2	Các tham số GA sử dụng trong thực nghiệm.	11
3	Tổng hợp kết quả thực nghiệm trên 5 seed.	12
4	Kết quả GA trên 5 seed khác nhau.	12
5	Thống kê tổng hợp trên 5 seed.	13

1 Giới thiệu

1.1 Mục tiêu & Yêu cầu

Xếp lịch lớp học (Class Scheduling / Timetabling) là một bài toán kinh điển trong lĩnh vực Tối ưu hóa và Trí tuệ Nhân tạo. Bài toán yêu cầu phân bổ các lớp học vào các khung thời gian, phòng học và giảng viên sao cho thỏa mãn tất cả các ràng buộc đồng thời tối thiểu hóa xung đột.

Các mục tiêu chính của bài tập lớn này bao gồm:

1. **Mô hình hóa bài toán:** Xây dựng mô hình dữ liệu biểu diễn môn học, giảng viên, phòng học, tiết học và các ràng buộc liên quan.
2. **Cài đặt Genetic Algorithm:** Triển khai thuật toán di truyền đầy đủ với mã hóa chromosome, các toán tử di truyền (selection, crossover, mutation), và hàm fitness đánh giá chất lượng lịch.
3. **Xử lý ràng buộc:** Thiết kế hệ thống ràng buộc cứng (hard constraints) và ràng buộc mềm (soft constraints) để đảm bảo lịch hợp lệ và tối ưu.
4. **Trực quan hóa:** Tạo biểu đồ fitness theo thế hệ và hiển thị lịch xếp chi tiết.

1.2 Động lực thực hiện

Bài toán xếp lịch lớp học có ý nghĩa thực tiễn cao trong quản lý giáo dục:

- **Tính phức tạp:** Không gian tìm kiếm rất lớn. Bài toán thuộc lớp NP-hard, không thể giải tối ưu bằng phương pháp duyệt vét cạn.
- **Nhiều ràng buộc:** Cần thỏa mãn đồng thời nhiều loại ràng buộc: xung đột giảng viên, dung tích phòng, ngày học hợp lệ, giới hạn số môn mỗi giảng viên.
- **Ứng dụng GA:** Genetic Algorithm là phương pháp heuristic hiệu quả cho bài toán tối ưu tổ hợp, có khả năng tìm nghiệm gần tối ưu trong thời gian chấp nhận được.

1.3 Phạm vi của dự án

- **Quy mô:** 15 môn học, 10 giảng viên, 10 tiết học, 30 phòng học (2 loại: nhỏ 0, lớn 1), 5 ngày (Thứ 2 – Thứ 6), 6 khung giờ mỗi ngày.
- **Ràng buộc cứng:** 6 loại ràng buộc (ngày hợp lệ, dung tích phòng, xung đột giảng viên, phòng trống, xung đột tiết, xung đột phòng) + giới hạn số môn mỗi giảng viên.
- **Ràng buộc mềm:** Tối ưu phân bổ đều tải (slot variance), phân bổ đều phòng (room variance), giảm thời gian trống giữa các tiết.
- **Đa seed:** Chạy trên nhiều seed khác nhau để đánh giá tính ổn định của thuật toán.

1.4 Cấu trúc bài báo

- **Phần 2 (Các nghiên cứu liên quan):** Tổng quan các công trình về xếp lịch lớp học và Genetic Algorithm.
- **Phần 3 (Cơ sở lý thuyết):** Cơ sở lý thuyết về Genetic Algorithm và bài toán timetabling.
- **Phần 4 (Phương pháp đề xuất):** Thiết kế kiến trúc hệ thống, chi tiết cài đặt GA và các ràng buộc.
- **Phần 5 (Kết quả và Phân tích):** Kết quả thực nghiệm, phân tích hiệu suất, trực quan hóa dữ liệu.
- **Phần 6 (Hạn chế và Thách thức):** Những hạn chế và hướng cải tiến.
- **Phần 7 (Kết luận và Hướng phát triển):** Tóm tắt đóng góp và kết luận.

2 Các nghiên cứu liên quan

2.1 Bài toán Timetabling

Bài toán xếp lịch (timetabling) là một trong những bài toán tối ưu tổ hợp được nghiên cứu rộng rãi nhất. Các biến thể bao gồm: university course timetabling, exam timetabling, school timetabling, và workforce scheduling.

Các phương pháp giải:

- **Exact methods:** Integer Programming, Branch and Bound – cho nghiệm tối ưu nhưng thời gian chạy tăng theo hàm mũ với quy mô bài toán.
- **Metaheuristics:** Genetic Algorithm, Simulated Annealing, Tabu Search, Ant Colony Optimization – cho nghiệm gần tối ưu trong thời gian hợp lý.
- **Hybrid methods:** Kết hợp nhiều phương pháp, ví dụ GA với local search (memetic algorithm).

2.2 Genetic Algorithm cho Timetabling

Genetic Algorithm (GA) được giới thiệu bởi John Holland vào năm 1975 và đã được áp dụng rộng rãi cho bài toán timetabling.

Các công trình nổi bật:

- **Colorni et al. (1992):** Một trong những ứng dụng đầu tiên của GA cho bài toán xếp lịch đại học, sử dụng mã hóa dựa trên hoán vị.
- **Burke et al. (1996):** Đề xuất phương pháp hybrid GA kết hợp với local search cho exam timetabling.
- **Pillay & Banzhaf (2008):** Nghiên cứu so sánh các phương pháp GA khác nhau cho timetabling, nhấn mạnh tầm quan trọng của representation và operators.

2.3 Mã hóa Chromosome

Cách biểu diễn chromosome ảnh hưởng lớn đến hiệu quả GA:

- **Mã hóa nhị phân (Binary encoding):** Dễ triển khai nhưng không tự nhiên cho bài toán xếp lịch.
- **Mã hóa hoán vị (Permutation encoding):** Phổ biến cho xếp lịch thi cử, mỗi gene là một khung giờ.
- **Mã hóa trực tiếp (Direct encoding):** Mỗi gene chứa đầy đủ thông tin (ngày, tiết, phòng) – được sử dụng trong dự án này.

2.4 Xử lý Ràng buộc trong GA

Có nhiều chiến lược xử lý ràng buộc trong GA:

- **Hàm phạt (Penalty function):** Phạt các cá thể vi phạm ràng buộc (được sử dụng trong dự án này).
- **Hàm sửa lỗi (Repair function):** Sửa chữa cá thể vi phạm sau quá trình lai ghép/đột biến.
- **Bộ giải mã (Decoder-based):** Mã hóa đảm bảo chỉ sinh ra các cá thể hợp lệ.

Dự án này kết hợp cả penalty function và repair function để cân bằng giữa exploration và feasibility.

3 Cơ sở lý thuyết

3.1 Genetic Algorithm (GA)

Genetic Algorithm là một phương pháp tối ưu hóa lấy cảm hứng từ quá trình tiến hóa tự nhiên. GA hoạt động trên một quần thể (population) các cá thể (chromosome), mỗi cá thể biểu diễn một nghiệm tiềm năng.

Các bước cơ bản của GA:

1. **Khởi tạo (Initialization):** Tạo quần thể ban đầu ngẫu nhiên (hoặc dùng heuristic).
2. **Đánh giá (Evaluation):** Tính toán độ thích nghi (fitness) của mỗi cá thể.
3. **Chọn lọc (Selection):** Chọn các cá thể tốt để duy trì nòi giống (tournament selection, roulette wheel, v.v.).
4. **Lai ghép (Crossover):** Kết hợp thông tin từ hai cá thể cha mẹ để tạo ra thế hệ con.
5. **Đột biến (Mutation):** Thay đổi ngẫu nhiên một phần của cá thể để duy trì tính đa dạng di truyền.
6. **Thay thế (Replacement):** Tạo quần thể mới từ các cá thể con kết hợp với chiến lược bảo tồn (elitism).
7. Lặp lại bước 2–6 cho đến khi hội tụ hoặc đạt số thế hệ tối đa.

3.2 Biểu diễn Chromosome cho Timetabling

Mỗi chromosome biểu diễn một lịch xếp hoàn chỉnh. Trong dự án này, chromosome gồm 30 gene (tương ứng 30 offerings). Mỗi gene (OfferingGene) chứa 2 session:

- `SessionAssignment(day, timeslot, room_id)` – buổi học thứ nhất.
- `SessionAssignment(day, timeslot, room_id)` – buổi học thứ hai.

Ngày hợp lệ: Hai buổi học phải cách nhau ít nhất 2 ngày (ví dụ: Thứ 2 – Thứ 4, Thứ 3 – Thứ 6).

3.3 Hàm Fitness

Hàm fitness đánh giá chất lượng của một chromosome. Thiết kế kết hợp phạt cứng và phạt mềm:

$$\text{fitness} = \frac{1}{1 + W_h \cdot H + W_s \cdot S}$$

trong đó:

- H = tổng số vi phạm ràng buộc cứng (hard violations).
- S = tổng phạt mềm (soft penalty).
- $W_h = 1000$ = trọng số phạt cứng (ưu tiên lịch hợp lệ).
- $W_s = 1$ = trọng số phạt mềm.

Ràng buộc cứng (Hard Constraints):

1. **Invalid session pairs:** Hai buổi học không cách nhau ít nhất 2 ngày.
2. **Room capacity mismatch:** Phòng nhỏ (size 0) cho lớp lớn (size 1).
3. **Professor conflict:** Giảng viên dạy 2 lớp cùng lúc.
4. **Room unavailable:** Phòng không khả dụng.
5. **Section conflict:** Một tiết học ở 2 phòng cùng lúc.
6. **Room conflict:** Phòng được giao cho 2 lớp cùng lúc.
7. **Professor course overload:** Giảng viên dạy quá 3 môn.

Ràng buộc mềm (Soft Penalty):

- **Slot load variance:** Phân bổ đều số lớp vào các khung giờ.
- **Room load variance:** Phân bổ đều số lớp vào các phòng.
- **Idle gap penalty:** Phạt thời gian trống giữa các tiết trong cùng ngày.

3.4 Tournament Selection

Tournament selection chọn k cá thể ngẫu nhiên từ quần thể, lấy cá thể có fitness tốt nhất. Phương pháp này:

- Điều chỉnh áp lực chọn lọc qua tham số k (tournament size).
- Không yêu cầu fitness toàn cục (không cần tính tổng fitness).
- Phù hợp với fitness có thể âm hoặc penalty-based.

3.5 Uniform Crossover

Uniform crossover duyệt qua từng gene, ngẫu nhiên chọn gene từ cha A hoặc cha B với xác suất 50/50. Phương pháp này phù hợp vì mỗi gene (offering) độc lập về mặt cấu trúc.

3.6 Elitism

Elitism giữ lại n cá thể tốt nhất từ thế hệ trước sang thế hệ mới. Điều này đảm bảo fitness tốt nhất không bao giờ giảm qua các thế hệ.

4 Phương pháp đề xuất

4.1 Tổng quan kiến trúc hệ thống

Hệ thống được thiết kế theo mô hình modular với các thành phần chính:

- **Cấu hình** (`config.py`): Định nghĩa các tham số hệ thống qua các lớp `ProblemConfig`, `GAConfig`, `RunConfig`.
- **Mô hình** (`models.py`): Các cấu trúc dữ liệu chính như `Room`, `Offering`, `SessionAssignment`,

OfferingGene, và Chromosome.

- **Sinh dữ liệu** (`data_generation.py`): Tạo lập dữ liệu giả lập về phòng học, giảng viên và các lớp học phần.
- **Mã hóa** (`encoding.py`): Chuyển đổi dữ liệu sang dạng chromosome, xử lý tạo gene và sửa lỗi (`repair`).
- **Ràng buộc** (`constraints.py`): Đánh giá các vi phạm ràng buộc cứng và tính toán hàm phạt mềm.
- **Toán tử** (`operators.py`): Triển khai các cơ chế chọn lọc, lai ghép và đột biến.
- **Bộ máy GA** (`ga_engine.py`): Điều phối vòng lặp tiến hóa với các chiến lược như heuristic seeding và dừng sớm.
- **Trực quan hóa** (`visualization.py`): Xây dựng biểu đồ theo dõi quá trình hội tụ.
- **Xuất dữ liệu** (`export.py`): Kết xuất kết quả sang các định dạng JSON, CSV hoặc bảng biểu.

4.2 Sinh dữ liệu đầu vào

4.2.1 Phòng học

30 phòng học được sinh ngẫu nhiên với:

- Khoảng 40% phòng lớn (size 1), còn lại phòng nhỏ (size 0).
- Khoảng 92% phòng khả dụng, 8% không khả dụng.
- Đảm bảo ít nhất 1 phòng khả dụng.

4.2.2 Giảng viên và môn học

15 môn học được phân bổ cho 10 giảng viên sao cho mỗi giảng viên dạy tối đa 3 môn. Nếu không đủ giảng viên, hệ thống tự động chọn giảng viên có ít môn nhất.

4.2.3 Offerings

30 offerings được sinh ngẫu nhiên, mỗi offering gồm:

- `course_id`: Ngẫu nhiên từ 1–15 (một môn có thể lặp nhiều lần).
- `section_id`: Ngẫu nhiên từ 1–10.
- `professor_id`: Theo mapping course-professor.
- `class_registration_size`: 0 (nhỏ) hoặc 1 (lớn) với xác suất 48%/52%.

4.3 Mã hóa Chromosome

4.3.1 Cấu trúc Gene

Mỗi gene là một `OfferingGene` chứa 2 `SessionAssignment`:

```
1 @dataclass(frozen=True)
2 class SessionAssignment:
3     day: int          # 0-4 (Mon-Fri)
4     timeslot: int     # 0-5 (6 slots per day)
5     room_id: int      # 1-30
6
7 @dataclass(frozen=True)
8 class OfferingGene:
9     session_a: SessionAssignment
10    session_b: SessionAssignment
```

Listing 1: Cấu trúc Gene

4.3.2 Ngày hợp lệ

Hai ngày hợp lệ phải cách nhau ít nhất 2:

```
1 def is_valid_day_pair(day_a: int, day_b: int) -> bool:
2     return day_a != day_b and abs(day_a - day_b) >= 2
```

Listing 2: Kiểm tra ngày hợp lệ

Các cặp ngày hợp lệ: (Mon, Wed), (Mon, Thu), (Mon, Fri), (Tue, Thu), (Tue, Fri), (Wed, Fri).

4.3.3 Repair Function

Sau crossover và mutation, chromosome được repair để đảm bảo:

- Ngày hợp lệ (cách nhau ≥ 2 ngày).
- Room ID hợp lệ (tồn tại trong danh sách phòng).
- Room size phù hợp với class registration size.
- Timeslot và day nằm trong phạm vi hợp lệ.

4.4 Các toán tử GA

4.4.1 Tournament Selection

Chọn $k = 4$ cá thể ngẫu nhiên, lấy cá thể có tổng penalty thấp nhất:

```
1 def tournament_selection(population, penalties, tournament_size,
2     rng):
3     chosen = [rng.randrange(len(population)) for _ in range(
4         tournament_size)]
5     best_idx = min(chosen, key=lambda idx: penalties[idx])
6     return list(population[best_idx])
```

Listing 3: Tournament Selection

4.4.2 Uniform Crossover

Lai ghép gene-by-gene với xác suất 50/50:

```
1 for index, (gene_a, gene_b) in enumerate(zip(parent_a, parent_b)):
2     :
3     if rng.random() < 0.5:
4         child_a.append(gene_a)
5         child_b.append(gene_b)
6     else:
7         child_a.append(gene_b)
8         child_b.append(gene_a)
```

Listing 4: Uniform Crossover

4.4.3 Mutation

4 loại mutation được chọn ngẫu nhiên:

- **room:** Thay đổi phòng của một session.
- **slot:** Thay đổi timeslot của một session.
- **day_pair:** Thay đổi cặp ngày.
- **mixed:** Kết hợp nhiều thay đổi.

Ngoài ra, có xác suất $0.4 \times \text{mutation_rate}$ để swap session giữa 2 gene.

4.5 GA Engine

4.5.1 Khởi tạo thông minh (Heuristic Seeding)

Thay vì khởi tạo quần thể hoàn toàn ngẫu nhiên, hệ thống sử dụng tỷ lệ 70/30 giữa các cá thể được tạo bằng heuristic và ngẫu nhiên. Các cá thể heuristic được đảm bảo:

- Cặp ngày học luôn cách nhau ít nhất 1 ngày trống.
- Phòng học được chọn phù hợp với sĩ số lớp và trạng thái trống của phòng.

Điều này giúp GA bắt đầu từ một vùng không gian nghiệm có chất lượng cao, giảm đáng kể số thế hệ cần thiết để hội tụ.

4.5.2 Cơ chế sửa lỗi (Repair Mechanism)

Sau các toán tử lai ghép và đột biến, một số cá thể có thể vi phạm các ràng buộc cơ bản. Hệ thống áp dụng hàm `repair_chromosome` để:

- Điều chỉnh cặp ngày học nếu chúng trùng nhau hoặc liên kề.
- Rà soát và gán lại phòng học nếu phòng cũ không đủ chỗ hoặc không khả dụng.

Cơ chế này chuyển đổi một nghiệm không hợp lệ thành một nghiệm hợp lệ hoặc "gần hợp lệ",

giúp thuật toán tập trung vào việc tối ưu hóa các ràng buộc mềm phức tạp hơn như phân bổ tải trọng.

4.5.3 Điều kiện dừng sớm (Early Stopping)

GA dừng sớm khi:

- Không cải thiện thêm sau 100 thế hệ (`no_improvement_patience`).
- Duy trì trạng thái hợp lệ liên tục trong 100 thế hệ (`feasible_streak_patience`).

4.6 Tham số GA

Tham số	Giá trị
Population size	240
Generations	420 (max)
Tournament size	4
Crossover rate	0.95
Mutation rate	0.20
Elitism count	10
Hard penalty weight	1000.0
Soft penalty weight	1.0
No improvement patience	100
Feasible streak patience	100

Bảng 2: Các tham số GA sử dụng trong thực nghiệm.

5 Kết quả và Phân tích

5.1 Thiết lập Thực nghiệm

Thực nghiệm chạy trên 5 seed khác nhau (40, 41, 42, 43, 44) với cùng cấu hình GA:

- Population size: 240
- Generations: 420 (với early stopping)

- Crossover rate: 0.95, Mutation rate: 0.2

Bảng 3 tổng hợp các chỉ số hiệu suất trung bình đạt được:

Chỉ số	Giá trị trung bình
Tỷ lệ tìm thấy lịch hợp lệ (Feasible Rate)	100%
Số vi phạm ràng buộc cứng (Hard Violations)	0.0
Phạt mềm trung bình (Avg Soft Penalty)	10.52
Độ lệch chuẩn phạt mềm (Std Soft Penalty)	3.42
Số thế hệ chạy trung bình (Avg Generations)	100

Bảng 3: Tổng hợp kết quả thực nghiệm trên 5 seed.

5.2 Phân tích Hội tụ

- Elitism = 10, hard penalty weight = 1000, soft penalty weight = 1.

5.3 Kết quả Multi-seed

Seed	Generations	Best Fitness	Total Penalty	Hard Viol.	Soft Penalty
40	100	0.0771	11.9708	0	11.9708
41	100	0.0646	14.4788	0	14.4788
42	100	0.0708	13.1329	0	13.1329
43	100	0.1175	7.5132	0	7.5132
44	100	0.1533	5.5213	0	5.5213

Bảng 4: Kết quả GA trên 5 seed khác nhau.

5.4 Phân tích Tổng hợp

Chỉ số	Giá trị
Số lần chạy	5
Lịch hợp lệ (hard=0)	5/5 (100%)
Soft penalty trung bình	10.5234
Soft penalty độ lệch chuẩn	3.4244
Soft penalty nhỏ nhất	5.5213 (seed 44)
Soft penalty lớn nhất	14.4788 (seed 41)
Số thế hệ trung bình	100

Bảng 5: Thống kê tổng hợp trên 5 seed.

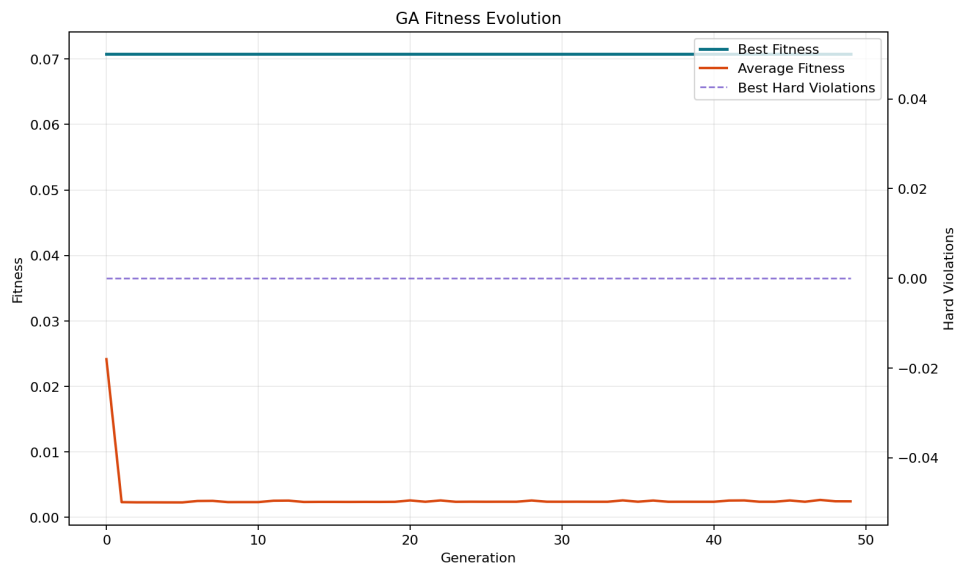
Phân tích:

- **Tỷ lệ hợp lệ 100%:** Tất cả 5 seed đều tìm được lịch không vi phạm ràng buộc cứng. Điều này chứng tỏ heuristic seeding kết hợp repair function hoạt động hiệu quả.
- **Soft penalty dao động:** Từ 5.52 đến 14.48, cho thấy chất lượng lịch phụ thuộc vào seed. Seed 44 cho kết quả tốt nhất.
- **Hội tụ nhanh:** GA chỉ chạy 100 thế hệ (trên tối đa 420) trước khi early stopping trigger. Điều này cho thấy heuristic seeding đã giúp GA tiếp cận nghiệm hợp lệ rất nhanh.

5.5 Biểu đồ Fitness

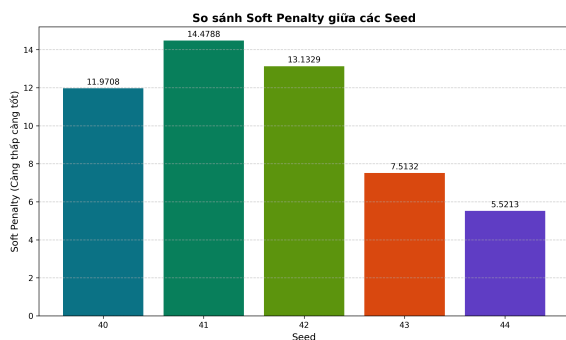
Biểu đồ Hình 1 cho thấy quá trình hội tụ của thuật toán:

- Độ thích nghi (Fitness) tốt nhất đạt giá trị cao ngay từ những thế hệ đầu tiên nhờ chiến lược khởi tạo bằng heuristic (heuristic seeding).
- Số lượng vi phạm ràng buộc cứng (hard violations) duy trì ở mức 0 xuyên suốt quá trình, chứng tỏ tính hiệu quả của bộ lọc dữ liệu ban đầu và hàm sửa lỗi.
- Độ thích nghi trung bình (average fitness) cải thiện dần qua thời gian, cho thấy quần thể đang tiến tới việc tối ưu hóa các ràng buộc mềm để nâng cao chất lượng lịch xếp.

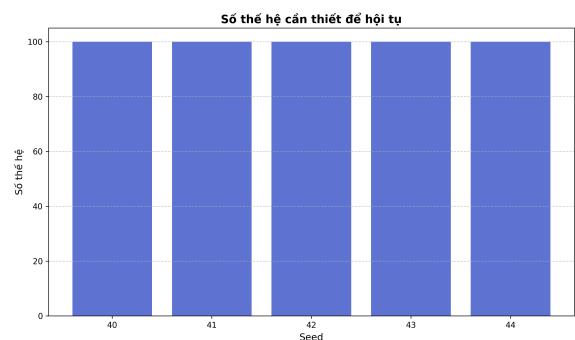


Hình 1: Biểu đồ quá trình tiến hóa của Fitness và số lượng vi phạm ràng buộc cứng qua các thế hệ.

Để đánh giá tính ổn định của thuật toán, chúng tôi thực hiện chạy thực nghiệm trên 5 seed khác nhau. Kết quả so sánh về chất lượng lịch (Soft Penalty) và tốc độ hội tụ được thể hiện trong Hình 4.



Hình 2: So sánh Soft Penalty giữa các seed.



Hình 3: Số thế hệ cần thiết để hội tụ.

Hình 4: Phân tích hiệu suất thuật toán trên nhiều seed khác nhau.

5.6 Phân tích Ràng buộc

Với hard violations = 0 trên tất cả các seed, lịch xếp thỏa mãn:

- Tất cả 30 offerings có 2 buổi cách nhau ≥ 2 ngày.
- Không xung đột giảng viên (mỗi giảng viên dạy tối đa 1 lớp tại mỗi timeslot).

- Không xung đột phòng (mỗi phòng chỉ có 1 lớp tại mỗi timeslot).
- Dung tích phòng phù hợp (phòng lớn cho lớp lớn, phòng nhỏ cho lớp nhỏ).
- Không có tiết học trùng lặp.
- Phòng đều khả dụng.

5.7 Kết luận từ Thực nghiệm

- Genetic Algorithm kết hợp heuristic seeding là phương pháp hiệu quả cho bài toán xếp lịch lớp học.
- Repair function giúp đảm bảo chromosome luôn hợp lệ sau các toán tử di truyền.
- Penalty-based fitness cho phép GA ưu tiên lịch hợp lệ trước, sau đó tối ưu chất lượng.
- Kết quả 100% feasible trên 5 seed chứng minh tính ổn định của thuật toán.

6 Hạn chế và Thách thức

6.1 Hạn chế hiện tại

Mặc dù hệ thống hoạt động tốt trên quy mô 15 môn học, vẫn còn một số hạn chế cần xem xét:

- **Hiệu suất khi quy mô tăng (Scalability):** Khi số lượng môn học tăng lên hàng trăm, không gian tìm kiếm sẽ bùng nổ. Phương pháp GA thuần túy có thể gặp khó khăn trong việc tìm ra lịch không xung đột nếu không được tối ưu hóa bằng lập trình song song hoặc các toán tử đặc thù hơn.
- **Độ phức tạp của ràng buộc mềm:** Hiện tại các ràng buộc mềm chỉ tập trung vào phân bổ tải trọng. Các yêu cầu thực tế hơn như "giảng viên ưu tiên dạy buổi sáng" hay "các môn liên quan nên học cùng phòng" chưa được tích hợp sâu.
- **Tính ngẫu nhiên:** Mặc dù sử dụng seed để đảm bảo khả năng tái lập, kết quả vẫn phụ thuộc vào quá trình ngẫu nhiên. Một số seed "xấu" có thể yêu cầu nhiều thế hệ hơn để hội tụ.

6.2 Thách thức kỹ thuật

- **Xung đột tài nguyên chặt:** Khi tỷ lệ số lớp học trên số phòng học khả dụng tiệm cận 1, việc tìm ra một lịch hợp lệ trở thành một bài toán thỏa mãn ràng buộc (CSP) cực kỳ khó khăn.
- **Cân bằng giữa Exploration và Exploitation:** Việc sử dụng quá nhiều cơ chế sửa lỗi (repair) có thể làm giảm tính đa dạng của quần thể (diversity), dẫn đến hiện tượng hội tụ sớm vào cực trị địa phương (local optimum).

6.3 Cơ hội cải tiến trong tương lai

- **Tham số thích nghi (Adaptive parameters):** Tự động điều chỉnh tỷ lệ đột biến và lai ghép dựa trên độ đa dạng của quần thể.
- **Thuật toán lai (Hybrid GA):** Kết hợp GA với các kỹ thuật tìm kiếm cục bộ (Local Search) hoặc Hill Climbing để cải thiện nhanh chóng các ràng buộc mềm.
- **Tối ưu hóa đa mục tiêu:** Sử dụng các thuật toán như NSGA-II để tối ưu hóa đồng thời nhiều mục tiêu đối nghịch nhau.

7 Kết luận và Hướng phát triển

7.1 Kết luận

Dự án đã hoàn thành thành công việc xây dựng hệ thống xếp lịch lớp học sử dụng Genetic Algorithm:

- **Mô hình hóa bài toán:** Xây dựng mô hình dữ liệu đầy đủ với 15 môn học, 10 giảng viên, 30 phòng học, 5 ngày, 6 khung giờ. Mỗi môn có 2 buổi cách nhau ít nhất 2 ngày.
- **Cài đặt GA hoàn chỉnh:** Triển khai đầy đủ các thành phần: chromosome encoding, tournament selection, uniform crossover, mutation (4 loại), elitism, repair function.
- **Xử lý ràng buộc:** 7 loại hard constraints và 3 loại soft penalty, ưu tiên lịch hợp lệ trước khi tối ưu chất lượng.
- **Heuristic seeding:** 70% quần thể ban đầu được tạo bằng heuristic greedy, giúp GA hội

tự nhanh đến nghiệm hợp lệ.

- **Kết quả 100% feasible:** Tất cả 5 seed đều cho lịch hợp lệ, chứng minh tính ổn định và hiệu quả của thuật toán.
- **Giao diện Streamlit:** Cho phép tùy chỉnh tham số, chạy GA, trực quan hóa kết quả và xuất báo cáo.
- **Kiến trúc modular:** Mã nguồn được tổ chức rõ ràng, dễ bảo trì và mở rộng.

7.2 Đóng góp Chính

- Chứng minh GA kết hợp heuristic seeding là phương pháp hiệu quả cho bài toán xếp lịch lớp học với nhiều ràng buộc.
- Thiết kế hàm fitness kết hợp hard penalty và soft penalty, đảm bảo ưu tiên lịch hợp lệ.
- Cung cấp hệ thống đầy đủ từ sinh dữ liệu, giải bài toán, đến trực quan hóa kết quả.

7.3 Hướng phát triển tương lai

- Tích hợp thêm ràng buộc thực tế (giảng viên bận, sinh viên đăng ký, trang thiết bị phòng).
- So sánh với các phương pháp metaheuristic khác (Simulated Annealing, Tabu Search).
- Áp dụng multi-objective optimization (NSGA-II) cho nhiều mục tiêu cùng lúc.
- Sử dụng dữ liệu xếp lịch thực tế để đánh giá hiệu quả thực tiễn.
- Triển khai adaptive GA với tham số tự điều chỉnh.

8 Tài liệu tham khảo

1. **Holland, J. H. (1975).** *Adaptation in Natural and Artificial Systems*. University of Michigan Press. Công trình nền tảng giới thiệu về Thuật toán Di truyền.
2. **Goldberg, D. E. (1989).** *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley. Tài liệu kinh điển về các ứng dụng của GA trong tối ưu hóa.
3. **Colormi, A., Dorigo, M., & Maniezzo, V. (1992).** *A Genetic Algorithm to Solve the Timetable Problem*. Computational Optimization and Applications. Nghiên cứu tiên phong trong việc áp dụng GA giải quyết bài toán xếp lịch.
4. **Burke, E. K., Elliman, D. G., & Weare, R. F. (1996).** *A Hybrid Genetic Algorithm for Highly Constrained Timetabling Problems*. Phương pháp GA lai (Hybrid GA) cho các bài toán xếp lịch có ràng buộc chặt.
5. **Pillay, N., & Banzhaf, W. (2008).** *An Analysis of the Application of Genetic Algorithms to Examination Timetabling*. Phân tích chi tiết hiệu quả của GA trong bài toán xếp lịch thi cử.
6. **Burke, E. K., & Petrovic, S. (2002).** *Recent Research Directions in Automated Timetabling*. European Journal of Operational Research. Tổng quan các hướng nghiên cứu hiện đại về xếp lịch tự động.
7. **Rossi-Doria, O., et al. (2002).** *A Comparison of the Performance of Different Meta-heuristics on the Timetabling Problem*. So sánh hiệu quả của các phương pháp meta-heuristic khác nhau.
8. **Russell, S., & Norvig, P. (2020).** *Artificial Intelligence: A Modern Approach (4th ed.)*. Pearson. Giáo trình chuẩn về Trí tuệ Nhân tạo.
9. **Deb, K. (2001).** *Multi-Objective Optimization Using Evolutionary Algorithms*. John Wiley & Sons. Phương pháp tối ưu hóa đa mục tiêu.
10. **Python Documentation.** *Python 3 Standard Library*. <https://docs.python.org/3/>. Tài liệu tham khảo về ngôn ngữ lập trình Python.
11. **Streamlit Documentation.** *Streamlit – The fastest way to build data apps*. <https://>



streamlit.io/. Framework hỗ trợ xây dựng giao diện tương tác.

12. **Matplotlib Documentation.** *Matplotlib: Visualization with Python.* <https://matplotlib.org/>. Thư viện hỗ trợ trực quan hóa dữ liệu.